A Tour of C++

Mihir Rao

Contents

1 The Basics **2**
- 1.1 Programs and compilation . . . 2
- 1.2 Functions . . . 2
- 1.3 Types, objects, and arithmetic . . . 3
- 1.4 Initialization and names . . . 3
- 1.5 Constants and compile-time evaluation . . . 4
- 1.6 Arrays, pointers, and references . . . 4
- 1.7 Tests and loops . . . 5
- 1.8 The memory model: copying and binding . . . 5

2 User-Defined Types **7**
- 2.1 Structures . . . 7
- 2.2 Classes and their interfaces . . . 8
- 2.3 Enumerations . . . 9
- 2.4 Unions and alternatives . . . 10

3 Modularity **12**
- 3.1 Declarations and definitions . . . 12
- 3.2 Separate compilation . . . 13
- 3.3 Namespaces . . . 15
- 3.4 Function arguments and return values . . . 16

Chapter 1: The Basics

C++ combines built-in operations, library components, and user-defined types. Functions and types organize direct access to memory and computation.

1.1 Programs and compilation

A compiler translates each source file into an object file. A linker combines object files and required library code into an executable for a particular platform. Source code can be portable across platforms even though a compiled executable usually is not.

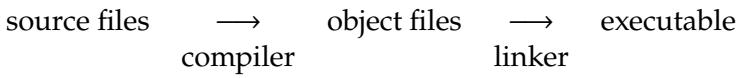


Figure 1: The two stages that turn C++ source into a program.

The standard library supplies facilities such as streams and containers. Its declarations must be made available before use. The familiar header form works widely; implementations that support the standard-library module can instead use `import std;`.

```
#include <iostream>

int main()
{
    std::cout << "Hello, C++!\n";
}
```

Execution begins at the program's single global `main` function. `std::cout` is the standard output stream, and « sends the string to it. The `std::` qualification identifies the library namespace. A zero return from `main`, including an implicit return at its end, indicates success. The compiler knows the type of every entity at its point of use, so it can check which operations are valid.

1.2 Functions

A function declaration states a name, a return type, and parameter types. A function definition also supplies the body. A call needs a visible declaration, and its arguments are initialized from the supplied expressions. `void` means that no value is returned.

```
double square(double x);                          // declaration
double square(double x) { return x*x; }           // definition
double area(double r) { return 3.14159*square(r); }
```

Breaking a computation into short, named functions makes its steps and dependencies visible. Overloading gives several functions the same name when their parameter types differ and their purpose is the same. The compiler chooses the best match for a call; a call with equally good matches is ambiguous and rejected.

1.3 Types, objects, and arithmetic

A type determines a set of values, permitted operations, and the representation of an object, a region of storage holding a typed value. A variable is a named object. Fundamental types include `bool`, `char`, `int`, and `double`. Their exact sizes depend on the implementation: `sizeof(char)` is 1, while `sizeof(int)` is often 4. An `unsigned` integer is primarily useful for bit operations, not as a general way to rule out negative numbers.

Integer literals are decimal unless marked otherwise: `0b1010` is binary, `0xA` is hexadecimal, and a leading `0` denotes octal. A decimal point or exponent makes a floating-point literal, as in `3.14` or `314e-2`. A single quote can separate digits without changing the value, as in `1'000'000`.

Arithmetic uses `+`, `-`, `*`, `/`, and, for integers, remainder `%`. Comparisons include `==` and `!=`; `=` assigns a value. The logical operators `&&`, `||`, and `!` differ from the bitwise operators `&`, `|`, `^`, and `~`. Mixed arithmetic can convert operands, for example an `int` to a `double`. Converting the result back to an integer can discard its fractional part. Keep expressions simple when conversions or evaluation order could obscure their meaning.

1.4 Initialization and names

Initialization creates an object with its first value; assignment replaces the value of an existing object. A brace initializer rejects narrowing conversions that would lose information:

```
double rate {2.5};
int a = 2.5;       // allowed, but loses the fractional part
int b {2.5};       // error: narrowing
auto count = 12;   // deduced as int
```

Prefer braces for a declared type and initialize each variable when it is introduced. `auto` asks the compiler to deduce a type from the initializer; it is especially helpful when the type is obvious or

cumbersome to spell. State the type when its range, precision, or meaning needs emphasis.

A name has a SCOPE, the region where it can be used. A local name lasts from its declaration to the end of its enclosing block. A class member belongs to class scope, and a namespace member belongs to namespace scope. The LIFETIME of an ordinary local object ends when its block exits; a namespace object lasts until program termination. A dynamically allocated object instead lasts until it is explicitly destroyed. Keep names in the smallest useful scope.

1.5 Constants and compile-time evaluation

`const` prevents modification through a name, even when its initial value is computed at run time. `constexpr` requires an initial value that the compiler can evaluate. A `constexpr` function may also run with ordinary run-time arguments, whereas a `consteval` function must be evaluated at compile time.

```
constexpr int cube(int x) { return x*x*x; }
consteval int twice(int x) { return 2*x; }

int input = 3;
const int observed = cube(input);   // runtime value, then immutable
constexpr int fixed = cube(3);     // compile-time value
int scaled = cube(input);          // valid runtime call
int error = twice(input);          // error: input is not constant
```

Constant expressions are required in places such as built-in array bounds and case labels. Immutability also makes ordinary code easier to reason about.

1.6 Arrays, pointers, and references

An ARRAY stores a fixed number of elements contiguously and uses zero-based indices. A POINTER holds an address; `&` obtains an object's address and `*` dereferences a pointer. In the following example, `p` points to the fourth element.

```
char letters[6] {'a', 'b', 'c', 'd', 'e', 'f'};
char* p = &letters[3];
char fourth = *p;                 // 'd'
```

A RANGE-FOR LOOP traverses all elements without an explicit index. Its loop variable can copy each element or refer to it:

index	0	1	2	3	4	5
letters	a	b	c	d	e	f

p = &letters[3]

Figure 2: A pointer identifies one element within a contiguous array.

```
int values[] {2, 4, 6};
for (auto x : values)  { std::cout << x << '\n'; } // copies
for (auto& x : values) { ++x; }                    // modifies elements
```

A REFERENCE is bound to an object at initialization and cannot be rebound. Passing a reference lets a function modify the caller's object without copying it; passing a `const` reference avoids a copy while promising not to modify it. A reference must refer to a valid object.

Use `nullptr` when a pointer intentionally has no target. Check it before dereferencing. A pointer to a zero-terminated character array may be advanced one element at a time until it reaches the terminating zero; this requires that the array really has such a terminator. `nullptr` is a pointer value, unlike the integer `0` used in older code.

1.7 Tests and loops

An `if` chooses a branch, a `switch` selects among distinct constant case labels, and `while` and `for` repeat work. `break` exits a loop or switch. A test of a number asks whether it is nonzero; a test of a pointer asks whether it is non-null. An `if` can declare a value whose scope stays within the statement:

```
std::vector<int> items {2, 4, 6};
if (auto n = items.size(); n != 0) {
    std::cout << n << " elements\n";
}
```

Choose a range-for loop for straightforward traversal and keep temporary names near their first use.

1.8 The memory model: copying and binding

Assignment copies a value. Copying an integer creates independent values; copying a pointer copies its address, so both pointers may refer to one object.

```
int x {2};
int y {3};
```

```
int* p {&x};
int* q {&y};
p = q;          // both pointers now point to y; x is unchanged

int& r {x};
int& s {y};
r = s;          // copies y's value into x; r still refers to x
```

The distinction between binding and assignment matters for references. `int& r{x}` initializes a reference by binding it to `x`; later `r = s` assigns through the reference. An uninitialized reference is not allowed. Function argument passing and return values also use initialization semantics.

Source. Bjarne Stroustrup, *A Tour of C++*, third edition (Pearson, 2023), Chapter 1, pp. 1–20.

Chapter 2: User-Defined Types

A user-defined type combines data and operations around a useful concept. Classes and enumerations supply abstraction with efficient representations.

2.1 Structures

A structure, declared with `struct`, groups related members. A simple vector representation pairs a pointer to elements with their count:

```
struct Vector {
    double* elem;
    int sz;
};

void vector_init(Vector& v, int s)
{
    v.elem = new double[s];
    v.sz = s;
}
```

The non-const reference lets `vector_init` change the caller's object. The expression `new double[s]` allocates an array on the free store, also called dynamic memory or the heap. That array outlives the scope that allocated it and requires matching destruction with `delete[]`. Its `double` elements must receive values before being read. Merely declaring `Vector v;` initializes neither member.

Assuming a positive size and successful input, the representation supports reading values and accumulating their sum:

```
double read_and_sum(int s)
{
    Vector v;
    vector_init(v, s);
    for (int i = 0; i != s; ++i)
        std::cin >> v.elem[i];

    double sum = 0;
    for (int i = 0; i != s; ++i)
        sum += v.elem[i];
```

```
    delete[] v.elem;
    return sum;
}
```

Use `.` to access a member through an object or reference and `->` through a pointer. Thus, for `Vector& rv` and `Vector* pv`, the size is accessed as `rv.sz` and `pv->sz`, respectively. Every caller must understand and preserve the relationship between the pointer and the size. Manual cleanup also becomes fragile when operations can fail. Prefer `std::vector` for ordinary programs; this small type exposes the design decisions behind a container.

2.2 Classes and their interfaces

A CLASS joins a representation to the operations that control its use. Its PUBLIC INTERFACE is available to callers; its PRIVATE MEMBERS hide implementation details. Members can be data, functions, or types. Separating interface from implementation allows the representation to change without changing every caller.

```
class Vector {
public:
    Vector(int s) : elem{new double[s]}, sz{s} { }
    double& operator[](int i) { return elem[i]; }
    int size() { return sz; }
private:
    double* elem;
    int sz;
};
```

This definition replaces the earlier structure. A CONSTRUCTOR has the class's name and initializes an object when it is created. Here, `Vector v(6);` calls `Vector(int)`. Its MEMBER INITIALIZER LIST, following the colon, initializes `elem` and `sz`; no separate initialization call is needed. There is no constructor taking zero arguments in this definition.

The SUBSCRIPT OPERATOR, `operator[]`, makes `v[i]` mean element access. Returning `double&` permits both reading and assignment. The `size()` function exposes the element count without allowing callers to change the stored size directly. Input and summation can therefore use `v[i]` and `v.size()` without naming the data members.

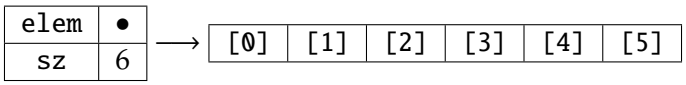


Figure 3: A vector handle stores a pointer and a count; the elements occupy a separate allocation.

The object in figure 3 is a fixed-size HANDLE to a run-time-sized array. The interface does not provide resizing.

> *Note.* This class illustrates initialization and access, but is not yet a complete owning container. It lacks a destructor to release the allocation, safe copy operations, size validation, and bounds checking. The constructor initializes the handle, not the values of the array elements. Use a standard-library container when these responsibilities must already be handled.

A `struct` is also a class: its members are public by default, whereas `class` members are private by default. Either can define constructors and other member functions. Access labels can appear in any order; placing the public interface first often makes a type easier to read.

2.3 Enumerations

An ENUMERATION provides named integer values within a distinct type. A SCOPED ENUMERATION, declared with `enum class`, keeps its ENUMERATORS in that type's scope and prevents implicit conversion to an integer or another enumeration.

```
enum class Color { red, blue, green };
enum class Traffic_light { green, yellow, red };

Color paint = Color::red;
auto signal = Traffic_light::red;
// Color bad = Traffic_light::red; // different types
// int n = Color::red;             // no implicit conversion
// Color bad = 2;                  // no implicit conversion
```

Unless values are specified, enumerators start at zero and increase by one. The UNDERLYING TYPE of an `enum class` defaults to `int`. Direct list initialization from an integer is allowed when the conversion to the underlying type is not narrowing. An explicit conversion can recover the integer value:

```
Color unnamed{5};                 // valid, but no named enumerator
int code = static_cast<int>(Color::red); // 0
```

Consequently, an enumeration value need not equal one of its named constants. Initialization, assignment, and comparisons are available, but arithmetic operations such as increment require a suitable operator definition.

```
Traffic_light& operator++(Traffic_light& t)
{
    using enum Traffic_light;
    switch (t) {
    case green:  return t = yellow;
    case yellow: return t = red;
    case red:    return t = green;
    }
    throw std::invalid_argument{"invalid traffic light"};
}
```

Here, prefix ++ advances the signal cyclically and returns a reference to the modified value. `using enum Traffic_light` makes the enumerator names available in the function's scope; otherwise each case and assignment would need a `Traffic_light::` qualifier. The final throw handles values without a named enumerator; `std::invalid_argument` is declared in `<stdexcept>`.

A PLAIN ENUMERATION, declared with `enum` without `class`, puts its enumerators in the surrounding scope and permits implicit integer conversions. For example, `enum Shade { light, dark };` allows `int n = dark;` and gives `n` the value 1. Prefer scoped enumerations to avoid name collisions and accidental mixing of values.

2.4 Unions and alternatives

A UNION overlays its members in the same storage. It holds one member's value at a time, with space sufficient for its largest member and any required alignment. When a symbol-table entry contains either a pointer or an integer, overlapping those alternatives can save space:

```
struct Node;
enum class Type { ptr, num };

union Value {
    Node* p;
    int i;
};

struct Entry {
    std::string name;
    Type t;
    Value v;
};
```

The TAG, also called a discriminant, records the active alternative. For an entry tagged `Type::num`, access `v.i`; for `Type::ptr`, access `v.p`. The language does not keep this tag synchronized with the union. Reading the wrong member is not a valid way to convert between these types. Encapsulating both the tag and union inside a class lets member functions maintain their correspondence.

A VARIANT manages the alternatives and tracks the active type. Using `std::variant` from <variant>, the entry can instead be represented and inspected as follows:

```
struct Entry {
    std::string name;
    std::variant<Node*, int> v;
};

void print_number(const Entry& e)
{
    if (std::holds_alternative<int>(e.v))
        std::cout << std::get<int>(e.v);
}
```

The predicate checks for the integer alternative before `std::get` retrieves it. This replaces the manually maintained tag and is usually safer and simpler than exposing a raw union.

Source. Bjarne Stroustrup, *A Tour of C++*, third edition (Pearson, 2023), Chapter 2, pp. 21–28 (PDF pages 34–41).

Chapter 3: Modularity

An interface states what a component provides; its implementation determines how it works. Explicit declarations and dependencies enable separate compilation.

3.1 Declarations and definitions

A declaration such as `double square(double);` gives a function's name, parameter type, and result type. Its definition supplies the body. Class members can likewise be declared inside the class and defined outside it:

```
class Vector {
public:
    Vector(int s);
    double& operator[](int i);
    const double& operator[](int i) const;
    int size() const;
private:
    double* elem;
    int sz;
};
```

The qualifier `Vector::` identifies an out-of-class definition as a member of `Vector`. A trailing `const` on a member function allows calls on a const object and prevents modification of the object's ordinary data members through that function. The two subscript overloads provide writable access for mutable objects and read-only access for const objects.

```
Vector::Vector(int s) : elem{new double[s]}, sz{s} { }
double& Vector::operator[](int i) { return elem[i]; }
const double& Vector::operator[](int i) const
{
    return elem[i];
}
int Vector::size() const { return sz; }
```

The class definition still contains its data representation even though its member-function bodies are elsewhere. As before, this introductory vector omits ownership management and error checks; use `std::vector` for a complete container. Compatible declarations can appear in multiple places.

An ordinary non-inline function must have one definition in the program; shared class definitions must remain consistent across translation units.

3.2 Separate compilation

Separate compilation lets a source file be compiled using the interfaces of other components. Their implementations can be compiled independently and linked into the program. Headers and modules express these boundaries in different ways.

3.2.1 Header files

A header file contains declarations intended to be included wherever an interface is needed. Place the preceding class definition in `Vector.h`. Both the implementation file and its clients include that same header, as in figure 4.

```
// Vector.cpp
#include "Vector.h"
// The out-of-class member definitions go here.
```

```
// user.cpp
#include "Vector.h"
#include <cmath>

double sqrt_sum(const Vector& v)
{
    double sum = 0;
    for (int i = 0; i != v.size(); ++i)
        sum += std::sqrt(v[i]);
    return sum;
}
```

Including the header in the implementation lets the compiler check that the definitions match their declarations. The const overloads above make the read-only accesses in `sqrt_sum` valid. The function assumes initialized, nonnegative elements. The <cmath> header declares `std::sqrt`.

A translation unit is a source file together with the contents brought in by its `#include` directives. Inclusion is textual: every translation unit processes its own copy of the header text. Header guards prevent repeated inclusion within one translation unit, but do not remove this work across translation units.

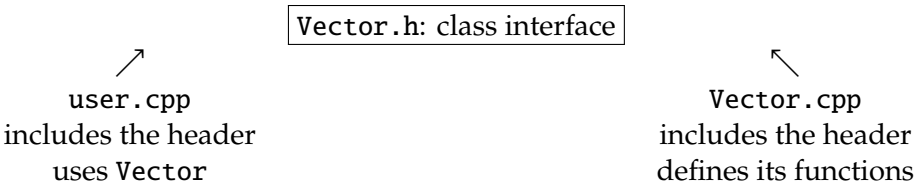


Figure 4: Client and implementation share a header and compile as separate translation units.

Headers can also expose dependencies unintentionally. Macros and declarations from one header can affect a later header, making inclusion order matter. Headers that include other headers can give clients accidental access to implementation details. Duplicated or conditionally altered declarations can produce inconsistent definitions. Share interfaces through headers rather than maintaining independent copies, and keep ordinary non-inline function bodies in implementation files.

3.2.2 Modules

A MODULE provides a language-supported boundary, introduced in C++20. A module interface is compiled for reuse; clients import it instead of textually including its source. `export module Vector;` names the module, and `export` identifies declarations that clients may use.

One way to organize the same vector is to put the class definition in an export block and its out-of-class member definitions afterward:

```
export module Vector;

export {
    // Put the complete Vector class definition here.
}
// Put its out-of-class member definitions here.

export bool operator==(const Vector& a, const Vector& b)
{
    if (a.size() != b.size())
        return false;
    for (int i = 0; i != a.size(); ++i)
        if (a[i] != b[i])
            return false;
    return true;
}
```

The exported class exposes its public interface; private members remain private. The separately exported equality operator compares sizes and elements.

A client can replace `#include "Vector.h"` with `import Vector;` and retain `#include <cmath>`. Headers and modules can coexist during migration. The compiler produces the module's importable interface from its exported declarations, as illustrated in figure 5; a separate handwritten header is unnecessary.

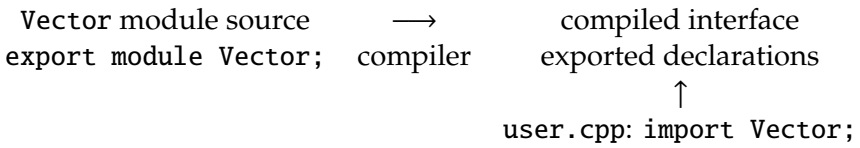


Figure 5: A module's compiled interface supplies the declarations used by importing clients.

Ordinary module imports do not inject header text or its macros into a client, removing the corresponding textual order dependencies. Importing a module does not automatically expose every declaration from its own imports; re-exporting a dependency requires an explicit interface decision. Declarations needed internally can therefore stay internal.

For example, a printing module can import library facilities and export only a printing function. A TEMPLATE parameter can make such a function work with vectors of different element types:

```
export module vector_printer;
import std;

export template<typename T>
void print(const std::vector<T>& values)
{
    std::cout << "{ ";
    for (const T& value : values)
        std::cout << value << ' ';
    std::cout << "}\n";
}
```

Importers gain access to `print`, without thereby gaining all the names from `std`. Module support and the standard-library module are separate requirements: `import std;` was not part of C++20. This example assumes a toolchain that supplies it. Reusing compiled interfaces can reduce build work substantially; the improvement depends on the program and toolchain.

3.3 Namespaces

A NAMESPACE groups related declarations and keeps their names from colliding with names in other components. Qualification with `::` selects the intended scope:

```cpp
namespace My_code {
    int run();
}

int My_code::run() { return 0; }

int main()
{
    return My_code::run();
}
```

The program's entry-point `main` belongs to the global namespace. Library facilities such as `std::cout` belong to `std`. Namespaces organize names; modules control which declarations a component exports.

A USING-DECLARATION introduces a selected name into the current scope. For example, `using std::swap;` permits calls written as `swap(x, y)` while other functions can still be explicitly qualified. A USING-DIRECTIVE, such as `using namespace std;`, makes the namespace's names available for unqualified lookup. Its broader effect can create ambiguity, so use it with care and do not put it in a shared header. A non-exported using-directive inside a module remains an implementation detail of that module.

3.4 Function arguments and return values

Arguments supply inputs; return values convey results. Prefer these interfaces to mutable globals, and keep shared state inside a suitable abstraction.

3.4.1 Argument passing

PASS BY VALUE initializes a local parameter from the argument; PASS BY REFERENCE binds the parameter to an existing object. A non-const reference permits modification of the caller's object. A const reference avoids copying while preventing modification through that reference.

```cpp
void test(std::vector<int> v, std::vector<int>& rv)
{
    v[1] = 99;
    rv[2] = 66;
}

std::vector<int> fib{1, 2, 3, 5, 8};
test(fib, fib);
// fib[1] is still 2; fib[2] is now 66.
```

Cheap-to-copy values are usually passed by value. Larger objects that need only to be read are usually passed by const reference:

```cpp
int sum(const std::vector<int>& values)
{
    int result = 0;
    for (int value : values)
        result += value;
    return result;
}
```

This avoids copying the vector while returning a small integer by value. Whether copying a type is cheap depends on its representation and the target machine; measure when the choice matters. Passing by value can also use a move when the argument permits it.

A DEFAULT ARGUMENT supplies an omitted argument at a call:

```cpp
void print(int value, int base = 10);
print(42);       // base 10
print(42, 16);   // base 16
```

This keeps one implementation for the same operation with a common setting. Overloading is useful when different argument types need different implementations of the same idea.

3.4.2 Value return

Return a result by value when the caller should receive its own object. Return a reference when granting access to an object that survives the call, such as an element of a still-living vector. Never return a reference or pointer to an ordinary local variable:

```cpp
int& bad()
{
    int value = 7;
    return value; // dangling: value is destroyed on return
}
```

Large results can also be returned by value. Copy elision constructs a result directly where it is needed, avoiding an intermediate copy. When elision is unavailable, a suitable move constructor can transfer resources cheaply instead of copying their contents.

```cpp
std::vector<int> make_values()
{
    std::vector<int> result{1, 2, 3, 5, 8};
    return result;
}

auto values = make_values();
```

The same applies to large matrix results. Prefer value results or named result structures to owning raw pointers and output parameters. A function can report failure with an exception.

3.4.3 Return type deduction and suffix return types

A function declared with `auto` can have its return type deduced from its return expression:

```cpp
auto mul(int i, double d) { return i*d; } // returns double
```

Deduction is convenient, especially for generic code, but couples the interface to the implementation: changing the expression can change the return type. Prefer an explicit type when a stable interface matters.

A suffix return type, also called a trailing return type, states the result after the parameter list. It is explicit rather than deduced:

```cpp
auto mul(int i, double d) -> double { return i*d; }
auto next_element() -> Element*; // assumes Element is declared
```

Placing the result after the parameters also allows return-type expressions to refer to parameter names. The leading `auto` signals that the return type comes later or will be deduced.

3.4.4 Structured binding

A function returns one value, but that value can contain several members. A named result type describes their relationship. A STRUCTURED BINDING gives local names to the parts of an object:

```
struct Entry {
    std::string name;
    int value;
};

Entry read_entry(std::istream& input)
{
    std::string name;
    int value{};
    if (!(input >> name >> value))
        throw std::runtime_error{"invalid entry"};
    return {name, value};
}

auto [name, value] = read_entry(std::cin);
```

Here the two binding names correspond, in order, to the two data members. The result is a single `Entry`; decomposing it does not require separate copies of its members. This example uses `<string>`, `<iostream>`, and `<stdexcept>`.

For an existing object, `auto [a, b] = entry;` creates an underlying copy, while `auto& [a, b] = entry;` binds to the original. Adding `const` and `&` gives read-only access without that copy. These choices also apply in range-for loops over a map:

```
for (const auto& [key, value] : counts)
    std::cout << key << ": " << value << '\n';

for (auto& [key, value] : counts)
    ++value;
```

Here `counts` is a `std::map<std::string, int>`: keys stay const while mapped values can change. Structured bindings also support arrays and tuple-like types, but not arbitrary private class data.

Source. Bjarne Stroustrup, *A Tour of C++*, third edition (Pearson, 2023), Chapter 3, pp. 29–42 (PDF pages 42–55).